

Project Report

Portfolio Application

Flutter + GetX Architecture Implementation

ID : C233173 Section : 6EM

1. Introduction

This report reflects on the design and development of a personal portfolio mobile application, built using Flutter and the GetX state-management framework. The project served as a practical, self-directed exercise that reinforced and applied several core concepts introduced in the Software Development II (SD2) lab course, including object-oriented design, layered software architecture, state management, reusable UI components, version control hygiene, and debugging of a real-world build/deployment issue.

2. Project Overview

The application presents a single-page portfolio: a profile header, an about section, a skills list with animated progress indicators, a projects showcase, and a contact section. It supports a light/dark theme toggle and includes size- and color-transition animations on multiple UI components (the theme toggle, the profile avatar, the skill bars, and the project cards). The codebase is organized using the GetX architecture, separating concerns into controllers, bindings, routes, data models, and presentation widgets.

3. How the Project Reflects Software Development 2 Lab Concepts

3.1 Object-Oriented Programming (OOP)

The project is built entirely around OOP principles taught in SD2. Data is encapsulated in model classes (SkillModel, ProjectModel) with immutable, well-typed fields. Controllers (PortfolioController, ThemeController) encapsulate state and behavior behind clear public interfaces, and the UI layer never manipulates that state directly — it only calls controller methods or reads reactive fields, which mirrors the encapsulation and abstraction principles emphasized in lab exercises on class design.

3.2 Layered / Modular Software Architecture

The lab course's emphasis on separating a system into layers (data, logic, presentation) is directly reflected in the folder structure:

- core/controllers — application state and business logic (the "logic" layer).
- core/bindings and core/routes — dependency wiring and navigation configuration.
- data/models — plain data classes representing domain entities.
- screens/ — presentation-only widgets that consume state reactively and contain no business logic.

This mirrors MVC/MVVM patterns discussed in SD2: the Model (SkillModel/ProjectModel and controller state), the View (widgets under screens/), and a Controller/ViewModel layer (GetX controllers) that mediates between them.

3.3 State Management and the Observer Pattern

GetX's reactive variables (.obs) and Obx() widgets are a practical implementation of the Observer design pattern covered in the lab's design-patterns module: widgets subscribe to state changes and re-render automatically without manual callback wiring, which is the same publish/subscribe relationship studied in the course's discussion of decoupled communication between components.

3.4 Dependency Injection

The AppBinding class registers controllers with Get.put() and injects them wherever needed via Get.find(). This models the dependency-injection concept from SD2's discussion of loose coupling — widgets depend on abstractions (the controller's public API) rather than constructing their own dependencies, which improves testability and separation of concerns.

3.5 Component Reusability

Widgets such as SectionTitle and _ContactRow were extracted once a repeated pattern was noticed across the About, Skills, Projects, and Contact sections. This reflects the DRY (Don't Repeat Yourself) principle and the emphasis on reusable, composable components stressed throughout the lab's UI-development exercises.

3.6 UI/UX and Animation Concepts

The lab course's coverage of user-experience fundamentals — feedback, affordance, and visual hierarchy — is applied through AnimatedContainer-based size and color transitions on the theme toggle, the avatar, the skill bars, and the project cards. These give the interface immediate visual feedback for user actions (tapping, pressing), which is a direct application of the responsive-UI principles discussed in lab sessions.

3.7 Version Control and Project Hygiene

A proper .gitignore and a clean, documented folder structure (with a README describing setup and architecture) reflect the lab's repeated emphasis on maintainable, shareable codebases rather than throwaway scripts — a practice explicitly graded in SD2 lab submissions.

4. Debugging and Problem-Solving

A significant real-world learning moment occurred when the exported project was reported as "incomplete" when run in Flutter. Diagnosing the issue reflected the SD2 lab's debugging methodology directly:

- **Reproduce and inspect:** the project archive was unpacked and its file tree compared against a known-good Flutter project structure.
- **Isolate the root cause:** two issues were found — stray directories created by a shell brace-expansion error, and missing platform runner folders (android/, web/) that Flutter requires to build and launch the app.
- **Apply a targeted fix:** the malformed directories were removed, and the missing android/ and web/ platform scaffolding (Gradle build files, AndroidManifest, MainActivity, launcher icons, web manifest and icons) was reconstructed by hand.
- **Verify:** the corrected archive was re-extracted and its structure re-checked before delivery, confirming the fix rather than assuming it worked.

This process — reproduce, isolate, fix, verify — is precisely the debugging discipline emphasized in the SD2 lab's troubleshooting sessions, and it reinforced why understanding a framework's underlying build system (not just its application code) matters in real software projects.

5. Conclusion

Building this portfolio application translated several Software Development 2 lab topics — OOP, layered architecture, state management via the observer pattern, dependency injection, component reuse, UI/UX feedback principles, and systematic debugging — into a single, cohesive, working application. Beyond reinforcing individual concepts, the project demonstrated how these principles combine in practice: a clean architecture made the codebase easier to extend, and disciplined debugging turned a confusing build failure into a clearly explainable, fixable problem.